Predwise – 2. Mülakat Teknik Görev (48 saat)

Amaç

Bir üretim tesisinde çalışan makinalara takılmış olan sensör kutusundan elde edilen ivme verilerinin aşağıdaki gereksinimleri karşılayacak şekilde bir sistem kurulması planlanmaktadır.

- yüksek frekansta veri toplanması,
- toplanan 10 saniyelik paketlere bölünmesi,
- (en az) %25 seviyesinde boyut küçültme,
- “hub” ortamına transfer edilmesi,
- hub’da ingest edilip DB’ye özet olarak yazılması,
- kullanıcıya özet + basit grafik olarak sunulması

Tasarlanan sistemin doğru mimari kararlarla, uçtan uca çalışacak şekilde tasarlanması beklenmektedir. Taski çözerken alınacak mimari kararların savunulabilir nitelikte olması önem arz etmektedir.

Verilenler

Size bir sensor_sim.py dosyası verilecektir. Bu modül;

- read_xyz() fonksiyonu ile her çağrıda (x,y,z) döndürmektedir (float).
- Makinadan alınan titreşim sinyallerine ek olarak gürültü ve farklı nedenlerden kaynaklanan anomaliler içerir.
- Makina zaman içinde değişen modlarda çalışmaktadır. Bu çalışma modları makinanın herhangi bir işletim sistemi (PLC vs) olmadığı için veri olarak çekilememektedir.

Teknik Gereksinimlar

Geliştirilecek sistemin aşağıdaki teknik gereksinimleri sağlaması beklenmektedir.

- Sistemde bir collector (edge/sensor box) ve bir hub olacaktır.
- Collector (Edge / Sensor Box) 3200 Hz hedef örnekleme ile veri toplar ve 10 saniyede bir paket oluşturur. Paket hazır olunca Hub’a gönderir.
- Hub (Ingest + Storage + API). Collector’dan paketlerini alır. Paketleri disk üzerinde saklar (cold storage). Kullanıcıya özet ve basit grafik sunar .
- Collector toplamda **en az 6 saatlik** veri üretecek şekilde çalıştırılabilmeli. Collector, paket tamamlanınca Hub’a göndermeli.

Yapılacaklar:

1- Tasarlanan mimari ve kod yapısı bir sunum olarak tasarlanıp mülakatta katılımcılara sunulmalıdır.
2- Tasarlanacak servisler mülakatı gerçekleştiren ekibe verilen termin tarihinden önce iletilmelidir.

Beklenen Mimari

İki ayrı servis/komponent:

(A) Collector (Edge / Sensor Box)

- 3200 Hz hedef örnekleme ile veri toplar.
- 10 saniyede bir paket oluşturur.
- Paketi sıkıştırıp yazar.
- Paket hazır olunca Hub'a gönderir.

(B) Hub (Ingest + Storage + API)

- Collector'dan sıkıştırılmış paketlerini alır.
- Paketleri disk üzerinde saklar (cold storage).
- DB'ye sadece **paket metadata + özet metrikler** yazar.
- Kullanıcıya özet ve basit grafik sunar.

1) Collector Gereksinimleri

1.1 Örnekleme

- Hedef: **3200 Hz**
- Paket boyu: **10 saniye** → **32.000 sample**
- Gerçek zamanlı timing kusursuz olmak zorunda değil.
    - Ama en azından **ölçülen efektif rate** raporlanmalı (ortalama Hz vb.)

1.2 Paket Formatı

Her pakette şu kolonlar olmalı:

- ts_ns (int64) — nanosecond timestamp
- x, y, z (float32)
- fs_hz (int32) — 3200
- seq (int64) — sample index (0..31999)

1.3 Süre / Veri Hacmi

Collector toplamda **en az 12 saatlik** veri üretecek şekilde çalıştırılabilmeli.

6 saatlik test için “hızlandırılmış simülasyon” kabul:
Örn. gerçek zamanlı 6 saat beklemek yerine “time dilation” ile hızlı üretim yapılabilir.
Bunu README’de açıkça belirtin.

1.4 Transfer

Collector, paket tamamlanınca Hub’a göndermeli.

- Tercih: HTTP multipart upload
- Retry mekanizması olmalı:
    - Hub down ise paket kaybolmasın
    - Basit backup yeterli

2) Hub Gereksinimleri

2.1 Ingest Endpoint

- Sıkıştırılmış veri dosyasını al
- validate et (kolonlar, fs_hz, n_samples)
- disk’e kaydet
- DB’ye özet yaz
- response:
- { "ok": true, "packet_id": "...", "n_samples": 32000 }

2.2 DB Yazımı

Ham sample’ların DB’ye tek tek yazılması **istenmiyor**.

DB’de en az şu tablo olsun:
packets

- Downsample metoduyla beraber paket bilgisini içeren metriklerin tabloda tutulması gerekir. Metriklerin ne olacağı size bırakılmıştır.

DB seçimi:

- SQL

2.3 Özet ve Grafikler

Grafikte ham 3200 Hz çizmek zorunda değilsiniz. Downsample/decimate beklenir.

3) Zorlayıcı Gerçek Hayat Şartları

Aşağıdakilerden **en az 3 tanesini** uygulayın:

1. **Idempotency**

- Aynı packet iki kez gelirse duplicate yaratma

2. **Backpressure**

- Hub yavaşsa Collector tarafında paket biriktirme + limit + loglama

3. **Data Optimization**

- Hamverinin DB üzerinde düşük boyutta tutulmasını sağlamak

4. **Observability**

- Structured log (json)
- basic metrics endpoint (metrics veya basit healthcheck)

5. **Data corruption check**

- SHA256 checksum ile payload doğrula

4) Teslim Formatı

Repo içinde:

- collector/ ve hub/ klasörleri
- README dosyası
    - nasıl çalıştırılır
    - mimari kararlar
    - 6 saatlik testi nasıl yaptınız (real-time mı hızlandırma mı)
    - hangi “zorlayıcı gerçek hayat” şartları yaptınız
- docker-compose.yml

5) Değerlendirme Kriterleri

- Uçtan uca çalışması (Collector → Hub → DB → Summary/Plot)
- Veri hacmini doğru yönetmesi
- Retry / idempotency gibi prod refleksi
- Kod okunabilirliği ve açıklama kalitesi

6) Kısıtlar

- Harici bulut servis yok (S3 vb. yok)
- Her şey lokal çalışabilmeli